

SAX парсер JSON представления "богатого" текста

Для обработки JSON представления "богатого" текста реализован SAX парсер специального вида (в отличие от парсера JSON общего вида). Данный парсер реализован на C++; поддерживает работу с форматом [JsonML](#). JSON представление "богатого текста" XML описано [здесь](#).

Парсер JsonRTSaxParser

Класс

```
1 sbis::jsonxml::JsonRTSaxParser
```

представляет собой JSON SAX парсер представления текста в формате Wasaby Rich Text.

Конструктор класса

```
1 JsonRTSaxParser::JsonRTSaxParser( UniquePtr< JsonRTSaxHandler >::type  
  rtf_handler );
```

принимает в качестве параметра умный указатель на обработчик событий типа JsonRTSaxHandler.

Метод

```
1 void Parse( String const& json );
```

принимает на вход строку JSON и обрабатывает ее, вызывая соответствующие callback'и обработчика, реализованные пользователем. Если входная строка не валидна, выбрасывается исключение, обработка прерывается.

Сценарии использования

Парсер не имеет графического интерфейса пользователя (GUI). При создании парсера указывается обработчик (хэндлер) типа **JsonRTSaxHandler**, реализующий колбэки, определяемые пользователем.

Типичный сценарий использования парсера понятен из рассмотрения следующего кода на C++:

```
1 String JsonXmlConverter::Convert( StringViewconst & json )  
2 {  
3     WStringBuilder xml;  
4     std::unique_ptr < JsonRTSaxHandler > xml_handler( new  
  XmlLikeJsonRTSaxXmlWritingHandler( xml ));  
5     JsonRTSaxParser parser( std::move( xml_handler ) );  
6     try  
7     {  
8         parser.Parse( json );  
9     }  
10    catch (...)  
11    {  
12        return String();  
13    }  
14    return xml.Get();
```

В данном примере создается обработчик handler типа **XmlLikeJsonRTSaxXmlWritingHandler** (он наследуется от **JsonRTSaxHandler**), создается парсер **parser** с этим обработчиком, на вход парсеру подается строка **json** для обработки, возвращается результат обработки, построенный обработчиком.

Обработчик JsonRTSaxHandler

Класс

```
sbis::jsonxml::JsonRTSaxHandler
```

представляет собой абстрактный класс — основу для обработчика событий для JSON SAX парсера **JsonRTSaxParser**. Обработчик реализуется пользователем путем реализации виртуальных методов — callback'ов для обработки следующих событий:

- начало элемента;
- конец элемента;
- атрибут (если атрибуты не агрегируются).

Набор методов, требующих реализации, зависит от параметра, задаваемого обработчику.

Конструктор класса

```
JsonRTSaxHandler( bool aggregate_attributes = true );
```

принимает в качестве параметра **aggregate_attributes** — "выполнять агрегацию атрибутов". Этот параметр имеет тип **bool** (по умолчанию "true").

Возможные значения типа:

- **false** -атрибуты обрабатываются по отдельности, каждый атрибут приходит в отдельном событии. Могут быть вызваны колбэки:
 - **OnElementBegin**(**StringView**, **StringView**, **std::map< String, String >**) — начало элемента. Аргументы — **type**, **value** и **attributes** (атрибуты всегда пустые).
 - **OnElementEnd**(**StringView**, **StringView**) — конец элемента. Аргументы — **type**, **value**;
 - **OnAttribute**(**StringView**, **StringView**) — атрибут. Аргументы — **key**, **value**.
- **true** — атрибуты объединяются вместе и приходят вместе с событием начала элемента. Могут быть вызваны колбэки:
 - **OnElementBegin**(**StringView**, **StringView**, **std::map< String, String >**) — начало элемента. Аргументы — **type**, **value** и **attributes**;
 - **OnElementEnd**(**StringView**, **StringView**) — конец элемента. Аргументы — **type**, **value**.

Методы класса JsonRTSaxHandler, реализуемые пользователем

OnElementBegin(3)

```
virtual bool OnElementBegin( StringView const& type, StringView const& value,
Map< String, String >::type const& attributes );
```

Функция обратного вызова для обработки начала элемента. Аргументы функции — **type**, **value** (текст или название тега) и **attributes**.

Название	Описание

type	Значение ключа "type" (тип элемента: тег или текст).
value	Значение ключа "value" (значение элемента: название тега или текстовая строка).
attributes	Набор атрибутов в виде словаря "ключ -> значение".

OnElementEnd(2)

```
virtual bool OnElementEnd( StringView const& type, StringView const& value );
```

Функция обратного вызова для обработки конца элемента. Аргументы функции — type, value.

Название	Описание
type	Значение ключа "type" (тип элемента: тег или текст).
value	Значение ключа "value" (значение элемента: название тега или текстовая строка).

OnAttribute(2)

```
virtual bool OnAttribute( StringView const& key, StringView const& value );
```

Функция обратного вызова для обработки атрибута. Аргументы функции — key, value.

Название	Описание
key	Название атрибута.
value	Значение атрибута.

OnMetaAttributes(1)



Примечание

Метод используется для обработки sabydoc-документов.

```

1 virtual bool OnMetaAttributes( JsonSabyDocSaxHandlerMetaAttributres
  const& /* attributes */ )
2     {
3         return true;
4     }
```

Функция обратного вызова для обработки атрибутов тега "meta" структуры документа SabyDoc. Аргументы функции — атрибуты тега "meta". На данный момент поддерживается только атрибут "tableOfContents".

Значения, возвращаемые методами

Каждый из методов возвращает значение bool — признак, следует ли продолжать дальнейшую обработку JSON (true или false).

Обработчик XmlLikeJsonRTSaxHandler

Класс

sbis::jsonxml::XmlLikeJsonRTSaxHandler

представляет собой другой абстрактный класс — возможно, более удобный для использования — также основу для обработчика событий для JSON SAX парсера JsonRTSaxParser. Представляет собой пример реализации JsonRTSaxHandler, предоставляющий другой набор callback'ов, подобный набору callback'ов обработчика XML парсера. Для использования обработчика данного типа пользователю потребуется переопределить три чисто виртуальных метода — callback'ов для обработки событий.

OnTagBegin(2)

```
1 virtual bool OnTagBegin( StringView const& name, Map< String, String
  >::type const& attributes ) = 0;
```

Функция обратного вызова для обработки начала тега с указанием названия и атрибутов.

Название	Описание
name	Название тега.
attributes	Набор атрибутов в виде словаря "ключ -> значение".

OnTagEnd(1)

```
1 virtual bool OnTagEnd( StringView const& name ) = 0;
```

Функция обратного вызова для обработки конца тега.

Название	Описание
name	Название тега.

OnText(1)

```
virtual bool OnText( StringView const& text ) = 0;
```

Функция обратного вызова для обработки текста.

Название	Описание
text	Текстовая строка.

Значения, возвращаемые методами

Каждый из методов возвращает значение bool — признак, следует ли продолжать дальнейшую обработку JSON (true или false).

Пример реализации XmlLikeJsonRTSaxHandler

В заключении приведем пример реализации обработчика событий для SAX парсера на основе XmlLikeJsonRTSaxHandler, выполняющий вывод данных в формате XML (детальная реализация собственно вывода XML (класс XmlWriter) для простоты опущена). Данный обработчик используется в конвертере JSON → XML.

```
1 class XmlLikeJsonRTSaxXmlWritingHandler : public
  XmlLikeJsonRTSaxHandler
2 {
```

```

3 private:
4     enum State : Int32
5     {
6         sBase,
7         sOpened
8     };
9
10 public:
11     XmlLikeJsonRTSaxXmlWritingHandler( WStringBuilder& xml ) // сюда
    XmlWriter будет выводить текст
12         : XmlLikeJsonRTSaxHandler()
13         , mWriter( xml )
14         , mState( sBase )
15     {
16     }
17     bool OnTagBegin( StringView const& name, Map< String, String >::type
const& attributes ) override
18     {
19         if( mState == sOpened )
20             mWriter.WriteTagOpenFinish(); // вывести >
21             mWriter.WriteTagOpenWithAttributes( name, attributes ); //
вывести <name attribute1="value1" attribute2="value2"
22             mState = sOpened;
23             return true;
24     }
25     bool OnTagEnd( StringView const& name ) override
26     {
27         if( mState == sOpened )
28             mWriter.WriteTagCloseImmediate(); // вывести />
29         else
30             mWriter.WriteTagClose( name ); // вывести </name>
31             mState = sBase;
32             return true;
33     }
34     bool OnText( StringView const& text ) override
35     {
36         if( mState == sOpened )
37             mWriter.WriteTagOpenFinish(); // вывести >
38             mWriter.WriteText( text ); // вывести text
39             mState = sBase;
40             return true;
41     }
42
43 private:
44     XmlWriter mWriter;
45     State mState;
46 };

```

Библиотека и заголовочные файлы

Для использования парсера следует подключить библиотеку **sbis-xml-json-converter** и включить заголовочный файл

```
#include <sbis-xml-json-converter/json_rt_sax_parser.hpp>
```



Исходный код библиотеки расположен в репозитории engine.